

TensorFlip:

A Fast Fully-Decentralized Computational Lottery for Cryptocurrency Networks

Aditya Ahuja

Department of Computer Science and Engineering, Indian Institute of Technology Delhi, India
 aahuja85@gmail.com / aditya.ahuja@cse.iitd.ac.in

Abstract—Distributed ledger technology, and specifically blockchain protocols, have been leveraged to define decentralized lotteries to circumvent challenges in traditional lottery systems. Unfortunately, lottery consensus in proof-of-resource (such as proof-of-work, proof-of-stake) blockchains is provably biased towards lottery users that command a majority of the resource determining consensus in their respective cryptocurrency networks, resulting in partial-decentralization. Further, classical consensus protocols are inherently slow in achieving consensus, and thus cannot be considered for designing fast decentralized lotteries.

We present a fast, fully-decentralized computational lottery TensorFlip, where the betting users and the lottery house are peers in the same cryptocurrency network, and the bet and winning per player is a consensus based on a rudimentary quantum and classical distributed computation. We employ a two-party verifiable random function construction for computing the lottery winnings. Our lottery operates in the presence of both fail-stop and Byzantine adversaries, under the synchronous network model. Although slower classical consensus based and partially-decentralized blockchain consensus based lotteries may exist, we believe we are the first to propose a quantum and classical computation based deterministic fully-decentralized lottery, with an expected constant round complexity protocol.

I. INTRODUCTION

A lottery is a recreational activity where the lottery players bet some assets, and are eligible for rewards, which are best case orders of magnitude greater than the asset value, from the lottery hosting company (equivalently referred to as the *lottery house*) [1]. Centralized lotteries have many disadvantages: they are slow, have monopolistic odds, and can introduce an uncertainty in the winning payoff for vested interests of the central authority [1], apart from being a single point of failure. Consequently, there is a need for *decentralized, provably fair, and fast* protocols for (computational) lotteries on digital assets, where the lottery house is elected from among the players, in order to construct a self-contained peer-to-peer distributed system. Apart from defining an autonomous decentralized lottery, such protocols can also be useful for wealth redistribution to reduce inequality [2], an initial coin offering [3], and decentralized fundraising [4].

Given the existing state of decentralized (crypto-)economies, cryptocurrency networks underlying permissionless blockchains [5] are best candidates for sustaining decentralized digital asset lottery protocols, given that users in such networks possess a digital wallet which makes digital asset transfers under a lottery more amenable, and that the said

cryptocurrency networks have an open nature of membership.

In this contribution, we address the following fundamental problem: *In cryptocurrency networks underlying proof-of-resource (PoR) blockchains (such as proof-of-work in Bitcoin [6] and proof-of-stake in Algorand [7]), given consensus resource empowered users, can we define a fast protocol, that, apart from recording each user's bet, can elect a single user as the lottery house provably fairly, in a consensus resource independent way?*

Unfortunately, existing permissionless PoR blockchain protocols fall short of defining a perfectly fair decentralized lottery, as is argued next.

Insufficiency of existing Proof-of-Resource Blockchains

The stated fundamental problem in the context of PoR blockchains reduces to arguing about the ‘degree of decentralization’ of consensus resources (such as mining power and stake) in the corresponding cryptocurrency networks. Recently, it has been proven that permissionless PoR blockchain protocols can never achieve full/perfect decentralization [8]. Consequently, using PoR blockchains alone, there can never be an unbiased lottery house election protocol as the consensus resource majority users can always dominate the agreement on the house election of a particular user.

The other alternative to consider is employing classical Byzantine fault tolerant protocols to fairly elect the house. As it is shown next, these protocols are probabilistic in agreement, and slow.

Insufficiency of Classical (Byzantine Agreement) Protocols

Given n lottery users, under a synchronous network with an adaptive Byzantine adversary¹, classical consensus protocols that achieve deterministic agreement, have an $\Omega(n)$ time complexity [9]. Randomized classical consensus protocols compromise on correctness for speed, by achieving probabilistic consensus, but their time complexity is still a function of n (more specifically $\Omega(\sqrt{\frac{n}{\log n}})$ [9]).

Having proposed an ambitious goal of designing fast and exact lotteries, we strive for an expected constant time complexity,

¹We will give details on the network model and threat model in the following section.

perfectly fair protocol with deterministic house agreement. As proposed next, TensorFlip achieves all the requisite goals for an optimum decentralized lottery.

Our Research Contribution

	consensus (agreement)	full decentralization	round complexity
Blockchain Consensus	probabilistic	impossible	$O(1)$
Classical Consensus	probabilistic	✓	$\Omega(\sqrt{\frac{n}{\log n}})$
TensorFlip	deterministic	✓	$O(1)$

Fig. 1. TensorFlip is technically superior to blockchain and classical consensus protocols for defining a decentralized lottery, given a synchronous network with adaptive fail-stop or Byzantine adversaries.

We present TensorFlip: an expected constant round complexity, deterministic, decentralized, lottery protocol. Our protocol operates over a synchronous network, in the presence of computationally bounded, adaptive, fail-stop or Byzantine fault inducing lottery users, the size of whom in the network is less than one-third of the total number of lottery users. Our protocol employs a simple quantum distributed protocol to achieve fast consensus by side-stepping the time lower-bound from its classical equivalent. We give the advantages of TensorFlip over existing consensus protocol families in Figure 1.

As an additional novelty, TensorFlip employs a two-party verifiable random function (VRF) for determining each user's lottery winnings². Here, the first party is an elected user in a lottery house role, and the second party is a user in a bet-placer role.

This paper is organized as follows. We present the preliminary definitions and constructions in Section II. We give an overview of the salient features of TensorFlip in Section III. We give the TensorFlip protocol for a fail-stop adversary over a synchronous network, in Section IV. Next, we give the TensorFlip protocol for a Byzantine adversary over a synchronous network, in Section V. We discuss the deployment of TensorFlip-like protocols on an asynchronous network in Section VI. Finally, we give the related work in Section VII, and conclude the paper in Section VIII.

II. PRELIMINARIES

We will assume the set of users in the TensorFlip cryptocurrency network is given by $[n] := \{1, 2, \dots, n\}$. We will systematically define the distributed and local computation properties that TensorFlip should achieve.

A. The Definition of Lottery Consensus

We will require that the TensorFlip protocol should achieve the following goals, rephrased appropriately from Byzantine

Agreement protocols [9]:

- *Agreement*: All honest users in $[n]$ agree on the bet from all honest users in $[n]$, and also agree on exactly one elected house h among all honest users in $[n]$, with probability 1.
- *Bet and House Validity*: If π_i^{bet} is a bet proposal from some honest user $i \in [n]$, then π_i^{bet} will be confirmed by all honest users in $[n]$. If a house proposal π_h^{hou} is confirmed by the honest users in $[n]$, then $h \in [n]$ is an honest user and π_h^{hou} is proposed by h .
- *Termination*: All honest users in $[n]$ always conclude on the bets and the house, with probability 1.

B. The Network (Communication) Model

We will assume that the TensorFlip cryptocurrency network is *synchronous*: there exists an upper-bound on the worst-case message latency in the network³, which implies that the communication between the users happens in well-defined *rounds*: a bounded period of time post which all messages (both quantum and classical) transmitted since the start of the period will be seen by all users.

C. The Threat (Fault) Model

We will assume that the adversary controls t users in $[n]$, where $t < \frac{n}{3}$. We will assume that the adversary is *computationally bounded*: can only leverage polynomial time algorithms for its malicious behaviour, *computationally separable*: can only leverage classical algorithms during the classical steps of the protocol, and leverage quantum algorithms during the quantum steps of the protocol, and *adaptive*: can change its strategy at each step of the protocol.

We also assume that the adversarial users have exactly one of the following two behaviours:

- *Fail-Stop*⁴: At any step in the protocol, based on the current state of the system, the adversary can stop/halt any t users in $[n]$ from sending messages and computing the lottery as defined in the protocol, with the intent of compromising the lottery consensus.
- *Byzantine*: At any step in the protocol, based on the current state of the system, the adversary can hijack/take-over any t users in $[n]$ to send arbitrary messages and compute arbitrarily even contrary to what is defined in the protocol, with the intent of compromising the lottery consensus.

³In the synchronous network model, there two separate notions of *strong synchrony*: where the message latency upper-bound is known; and *weak synchrony*: where the said bound is unknown. Our protocols operate in both these network sub-models.

⁴Some blockchain and distributed consensus literature refer to a fail-stop adversary as a crash fault adversary.

²Traditional VRFs require a single verifiable input to generate randomness.

D. Full Decentralization

It has been established [8], that owing to their proof-of-resource based consensus (where the resource may be hash power in proof-of-work blockchains or stake in proof-of-stake blockchains), permissionless blockchains, without a trusted third party, can never achieve *full decentralization*: the privilege that all users have exactly the same chance/probability to propose a block, independent of their resource power in the network. TensorFlip requires full decentralization for a perfectly fair house agreement.

E. Basics of Quantum Computing as relevant to Consensus

We give the three fundamental quantum computing principles [10], as they would be relevant to the quantum distributed computation component of the house agreement under TensorFlip.

- **Quantum State Superposition:** Given a quantum state $|\psi\rangle$, with probability amplitudes $\alpha_k \in \mathbb{C}$ ⁵, for any set of values for k , such that $|\psi\rangle := \sum_k \alpha_k |\psi_k\rangle$, the probability that the measurement of $|\psi\rangle$ results in ψ_k is $|\alpha_k|^2$.
- **Quantum State Collapse:** Given a quantum state $|\psi\rangle$, such that $|\psi\rangle := \sum_k \alpha_k |\psi_k\rangle$, the measurement of $|\psi\rangle$ results in ψ_k for some k , then all future measurements of $|\psi\rangle$ will result in ψ_k with probability 1.
- **Quantum State Entanglement:** Given a quantum state $|\psi\rangle$, such that $|\psi\rangle := \sum_k \alpha_k |\psi_k^A, \psi_k^B\rangle$, then, on measurement of $|\psi\rangle$, $\Pr[B \text{ sees } \psi_k^B | A \text{ sees } \psi_k^A] = 1$.

F. Basics of Cryptographic Sortition

We now present the cryptographic sortition (classical) algorithm, in Algorithm 1, which is inspired and reproduced from Algorand [7]. It requires a verifiable proof π , which, coupled with an ideal (under the random oracle model [7]) collision resistant hash function (CRHF) H , results in a binomial random variable over $\{0, 1, \dots, W\}$, for some input W as a member of π . More specifically, given $p \in [0, 1]$ and $W \in \mathbb{Z}_+$ are verifiable under π , $\forall z \in \{0, 1, \dots, W\}$, $\Pr[z \text{ tokens are won}] = \binom{W}{z} p^z (1-p)^{W-z}$, $\mathbf{E}[z] = pW$, and $\max(z) = W$.

The TensorFlip protocol will leverage this sortition routine to locally compute winnings of the bet-placers, given the bet space and house has been agreed upon.

III. OVERVIEW OF THE TENSORFLIP PROTOCOL

A. Notation

We will assume $\circ$ to be the (infix) bit string concatenation operator. We assume each user i in the cryptocurrency network $[n]$ is identified by its public verification key vk_i and signs

⁵We will use $\mathbb{C}$ to denote the set of complex numbers.

Algorithm 1 HashBin for User Winnings

Given:

$H : \{0, 1\}^* \rightarrow \{0, 1\}^{hlen}$ is an ideal CRHF

$\mathbf{B}(k; p, W) := \binom{W}{k} p^k (1-p)^{W-k}$

procedure HASHBIN(π, p, W)

1: $z \leftarrow 0$

2: **while** $\frac{H(\pi)}{2^{hlen}} \notin [\sum_{k=0}^z \mathbf{B}(k; p, W), \sum_{k=0}^{z+1} \mathbf{B}(k; p, W))$:

2.1: $z \leftarrow z + 1$

3: **return** z

end procedure

its bet and house proposals under a private signing key sk_i . Given a signature scheme (sign, ver), a message m and its signature by user i is given by $\text{Sig}_i(m)$, such that $\forall i, m, \text{ver}(vk_i, \text{Sig}_i(m)) = 1$. For a given user $i \in [n]$, we give user i 's signed bet proposal as π_i^{bet} , and two signed house proposals as π_{i,b_i}^{hou} for $b_i \in \{0, 1\}$. We will denote the concatenation of the bet with the b_i 'th house proposal by π_{i,b_i}^* , given a user i . We use a collision resistant hash function (CRHF) H , the image under which is uniformly distributed under the random oracle model [7], for any pre-image distribution. We use a binomial function $\mathbf{B}$ corresponding to the binomial distribution. Finally, we employ quantum states which are a concatenation of n qudits: each qudit is a bit-string represented as a quantum state. $\forall i \in [n]$, the i 'th qudit is forwarded to user i .

The summary of the notation used is given in Figure 2.

Symbol	Description
vk_i	Public verification key of user i
sk_i	Private signing key of user i
u_i	Number of digital tokens bet by user i
w_i	Digital token weight proposed by user i
$(\text{sign}, \text{ver})$	Signature scheme
$\text{Sig}_i(m) := (m, \text{sign}_{sk_i}(m))$	Message $m \in \{0, 1\}^*$, and its signature by user i
π_i^{bet}	Signed bet proposal by user i
π_{i,b_i}^{hou}	Signed house proposals by user i for $b_i \in \{0, 1\}$
$\pi_{i,b_i}^* := \pi_i^{\text{bet}} \circ \pi_{i,b_i}^{\text{hou}}$	Signed lottery proposals by user i for $b_i \in \{0, 1\}$
H	An ideal collision resistant hash function
$\mathbf{B}$	Binomial distribution
$ m, m, \dots, m\rangle_n$	A quantum state on n qudits with value m

Fig. 2. Summary of symbols used in the TensorFlip protocols.

B. Outline of the Lottery Agreement

Each user $i \in [n]$ prepares $\pi_i^{\text{bet}} := \text{Sig}_i(vk_i, \text{BET}, u_i)$, $\pi_{i,b_i}^{\text{hou}} := \text{Sig}_i(vk_i, \text{HOUSE}, w_i, f_i, b_i)$ (for some $f_i \in (0, 1)$, and $b_i \in \{0, 1\}$). TensorFlip achieves lottery agreement via three computation steps. The first step of TensorFlip, which we call *bet space agreement*, is for $[n]$ to agree on all bets $\{\pi_i^{\text{bet}}\}_{i \in [n]}$. The second step of TensorFlip, which we call *house agreement*, is for all users in $[n]$ to agree on exactly one house proposal π_{h,b_h}^{hou} out of a total of $2n$ house proposals, with two house proposals permitted per user in

$[n]^6$, under a provably uniform distribution on the set of all house proposals. Finally, after both the bet space and house agreements, TensorFlip relies on a local classical computation for each user in $[n]$ as a final step, to determine the winnings per user (details in Section III-C).

1) *Bet Space and House Agreement under a Fail-Stop Adversary*: Given a fail-stop adversary, TensorFlip achieves bet-space and house agreement via a quantum consensus protocol with the quantum states defined as a function of the bet and house proposals (which are essentially classical verifiable messages), corresponding to each user in the network.

2) *Bet Space and House Agreement under a Byzantine Adversary*: Given a Byzantine adversary, TensorFlip achieves bet space and house agreement in three steps: (i) Each user classically broadcasts (parallelly) their bet and house proposals; (ii) Each user agrees on the *lottery space*: which is a union of the bet space and the house space⁷ by executing a PoR blockchain consensus protocol; (iii) Each user agrees on a single house proposal by executing a quantum Byzantine agreement protocol.

After the bet space and house agreement, the users locally compute the winnings, as explained next.

C. Token Transfer and Computation of User Winnings

Token Transfer: We first explain how digital token transfers take affect under TensorFlip. On bet space agreement, there is an implicit escrow of each user's bets to the network: $\forall i \in [n], i \xrightarrow{\text{commit } u_i} [n]$. On house agreement, first the house collects the escrow: $[n] \xrightarrow{\text{transfer } \sum_i u_i} h$, and then each user's winnings are transferred to the respective user by the house: $\forall i \in [n], h \xrightarrow{\text{win } z_i} i$.

Winning Draw: We now explain how for each user, the user winnings are computed. Each user computes $\forall i \in [n]$, winning z_i of user i as a random variable on $\{0, 1, \dots, w_h \times u_i\}$ using HashBin (Algorithm 1) with proof $\pi_{h,b_h}^{\text{hou}} \circ \pi_i^{\text{bet}}$. Now, we show that each user i 's winnings are proportionate to its bet u_i and the elected house's token weight w_h . Under the random oracle model, as the hash image is uniformly distributed, user i claims $\mathbf{E}[z_i] = f_h w_h u_i$ tokens in expectation. So, the house h wins in expectation as long as $f_h w_h < 1$. Further, given that the house wishes to win on average, implying $1 < w_h < \frac{1}{f_h}$, in the best case the user claims $\max(z_i) = w_h u_i > u_i$. Note that the expected value of random variable z_i is a direct result from the binomial distribution employed in HashBin.

A toy model of TensorFlip execution is given in Figure 3.

⁶We introduce two house proposals per user to add non-determinacy, and freedom for the user to choose multiple weights w .

⁷Recall there are a total of $2n$ house proposals.



Fig. 3. TensorFlip toy model with a 9 user cryptocurrency network: (1) each user bets some digital tokens (indicated by the cryptocurrency bet); (2) the network runs the protocol for bet and house agreement; and (3) each user locally computes each user's winnings (indicated by the hand offering a cryptocurrency).

IV. FAIL-STOP TENSORFLIP UNDER SYNCHRONY

We give the first iteration of TensorFlip under a fail-stop adversary over a synchronous network. We show how our protocol achieves determinism in lottery agreement, full-decentralization on the house election, while terminating in constant time (without employing any blockchain protocol).

A. The Fail-Stop TensorFlip Distributed Algorithm

We give the TensorFlip fail-stop protocol TensorFlip-Sync-FS in Algorithm 2.

Algorithm 2 TensorFlip-Sync-FS Protocol

1: Bet Space and House Agreement:

1.1: $\forall i \in [n]$, user i prepares

$$|l_i\rangle := \frac{1}{\sqrt{2}} |\pi_{i,0}^*, \pi_{i,0}^*, \dots, \pi_{i,0}^*\rangle_n + \frac{1}{\sqrt{2}} |\pi_{i,1}^*, \pi_{i,1}^*, \dots, \pi_{i,1}^*\rangle_n,$$

and shares the j -th qudit of $|l_i\rangle$ with user j .

1.2: $\forall i \in [n]$, user i prepares

$$|v_i\rangle := \frac{1}{\sqrt{n^3}} \sum_{a \in [n^3]} |a, a, \dots, a\rangle_n,$$

and shares the j -th qudit of $|v_i\rangle$ with user j .

1.3: $\forall i \in [n]$, measure $|v_i\rangle$ in the standard basis to get v_i .

Elect house (leader) $h := \arg\max_{i \in [n]} v_i$

1.4: $\forall i \in [n] \setminus \{h\}$, measure $|l_i\rangle$ in any basis to get π_i^{bet} .

1.5: Measure $|l_h\rangle$ in the standard basis to get $\pi_h^{\text{bet}}, \pi_{h,b_h}^{\text{hou}}$.

2: (Local) Winning Draw: Each user computes $\forall j \in [n]$, winning $z_j \leftarrow \text{HashBin}(\pi_{h,b_h}^{\text{hou}} \circ \pi_j^{\text{bet}}, f_h, w_h \times u_j)$.

A clarification on bet-space agreement is due. In Steps 1.4 and 1.5, quantum measurement recovers π_i^{bet} with probability 1, as $\forall i \in [n], |l_i\rangle$ contains the bet proposal for user i in both sub-states as a part of $\pi_{i,0}^*$ and $\pi_{i,1}^*$ respectively. We would also like to state that Step-1 is a lottery agreement equivalent of the fail-stop quantum Byzantine agreement from [9].

B. Features of Fail-Stop TensorFlip

We give the relevant distributed consensus features of fail-stop TensorFlip in the following subsections.

1) *Deterministic Distributed Consensus*: Given the principle of quantum entanglement in Section II-E, on measurement of $|l_i\rangle$ or $|v_i\rangle$, whichever qudit is observed by user i will also be seen by user $j, \forall j \neq i$. This implies determinism in the lottery consensus on the observed output of the distributed computation.

2) *Full-Decentralization on the House*: We prove that the house user h is a uniformly distributed random variable over the set of users $[n]$. We also show that the house proposal bit b_h is uniformly distributed.

Theorem 1 (Fail-Stop TensorFlip Full-Decentralization): TensorFlip-Sync-FS achieves full-decentralization in electing $h \xleftarrow{\text{uniform}} [n]$, $b_h \xleftarrow{\text{uniform}} \{0, 1\}$.

Proof. From Steps 1.2 and 1.3 in TensorFlip-Sync-FS, in conjunction with the quantum superposition principle on probability amplitudes in Section II-E, it is easy to see that $\forall i \in [n], \forall a \in [n^3], \Pr[v_i = a] = \frac{1}{n^3}$. This implies that each house leader is equi-probable to get the highest vote. In Step 1.5, b_h is a uniformly distributed bit, as $\forall h \in [n], |l_h\rangle$ is defined to result in π_{h,b_h}^* with probability $\frac{1}{2}$, for each $b_h \in \{0, 1\}$. The result of the theorem follows. ■

3) *Expected Constant Round Complexity*: We show next that TensorFlip-Sync-FS terminates in three rounds or less, in expectation. Our argument is similar to the fail-stop quantum Byzantine agreement protocol's time complexity [9].

Theorem 2 (Fail-Stop TensorFlip Round Complexity): TensorFlip-Sync-FS with a fail-stop adversary with size at most $\frac{n}{3}$, achieves deterministic bet space and house agreement, in expectation, in three rounds or less.

Proof (re-interpreted from [9]). The adversary has to decide which users to stop before commencement of Step-1. Note that this is before the measurement of bet and house proposal quantum states. Since during Step-1 all the users are symmetric in their distributions, the best the adversary can do is to crash-halt up to $\frac{n}{3}$ players from transmitting all their messages during Steps 1.1 and 1.2. The adversary will succeed only if it happens to stop the user which is going to be the house leader, but this will happen with probability less than $\frac{1}{3}$. This implies that Step-1 on bet space and house agreement will succeed with probability $\geq \frac{2}{3}$. The probability of a tie for the house election is negligible (less than 4% for even a single adversarial user).

Given that all the messages upto Step-1.2 were delivered, the probability for any house proposal π_{h,b_h}^{hou} to be the outcome of the protocol for all the honest users is exactly $\frac{1}{2}$ for either bit b_h , so the probability that all the honest users will see π_{h,b_h}^{hou} is at least $\frac{1}{3}$.

By geometric distribution, this implies that in three rounds or

less in expectation, all honest users will agree on the same bet-space and house. Thus the theorem result follows. ■

Note that the message complexity is assumed to be approximately constant, since the message transmission is instantaneous (for instance, using nil quantum communication [11]).

V. BYZANTINE TENSORFLIP UNDER SYNCHRONY

We now give a second iteration of TensorFlip under a Byzantine adversary. Note that a Byzantine quantum adversary can do a key-space traversal in quantum polynomial time, for instance by using Grover's search [12]. Thus any protocol employing classical VRFs in the quantum agreement step is susceptible to forgery. Thus the need for an alternate protocol with the separation of quantum and classical information, for safety against a separable adversary.

A. The Byzantine TensorFlip Distributed Algorithm

We give the Byzantine TensorFlip protocol TensorFlip-Sync-Byz in Algorithm 4, that leverages quantum Byzantine agreement in Algorithm 3 (re-interpreted from [9]). We use PoRConsensus to represent any proof-of-resource blockchain consensus protocol (such as Bitcoin, Algorand, or RapidChain [5]).

Algorithm 3 QA-Sync-Byz for Byzantine House Agreement

Given:

$$\forall k, i \in [n], |v_i^k\rangle := \frac{1}{\sqrt{n}} \sum_{a \in [n]} |a, a, \dots, a\rangle_n$$

procedure QA-Sync-Byz($[n]$)

1: $\forall k \in [n], \forall i \in [n]$, user k prepares $|v_i^k\rangle$, and shares the j -th qudit of $|v_i^k\rangle$ with user j using QVSS [13].

2: $\forall k \in [n], \forall i \in [n]$, measure $|v_i^k\rangle$ in the standard basis to get v_i^k . Compute $v_i := \oplus_{k \in [n]} v_i^k$.

3: Elect house (leader) $h := \text{argmin}_{i \in [n]} v_i$.

4: Compute b_h as the parity bit of v_h .

5: **return** (h, b_h)

end procedure

Algorithm 4 TensorFlip-Sync-Byz Protocol

1: *House and Bet Proposal*: Each user $i \in [n]$ broadcasts $(\pi_i^{\text{bet}}, \pi_{i,0}^{\text{hou}}, \pi_{i,1}^{\text{hou}})$ over the classical channel.

2: *Lottery Space Agreement*:

$B^* \leftarrow \text{PoRConsensus}(\{(\pi_i^{\text{bet}}, \pi_{i,0}^{\text{hou}}, \pi_{i,1}^{\text{hou}})\}_{i \in [n]})$

3: *House Agreement*: $(h, b_h) \leftarrow \text{QA-Sync-Byz}([n])$

4: *(Local) Winning Draw*: Each user computes $\forall j \in [n]$, winning $z_j \leftarrow \text{HashBin}(\pi_{h,b_h}^{\text{hou}} \circ \pi_j^{\text{bet}}, f_h, w_h \times u_j)$.

B. A Brief Security Analysis

It is to be noted that Algorithm 3 leverages quantum verifiable secret sharing (QVSS) from [13] for sharing the vote qudits. A formal analysis of this construction is beyond the scope of

this work. We would only like to state that QVSS is needed to ensure uniform distribution on the vote quantum states, as a Byzantine adversary might try to change the distribution in its favour for house election.

There is a possibility of the adversary dropping some of the bet and/or house proposals during the agreement in Step-2, resulting in a lottery *sub-space* agreement. The reason this problem is circumvented is that there exists a honest majority in proof-of-resource blockchains [5], and the said honest users would make sure that all n bet proposals and all $2n$ house proposals make it on-chain. In future, we would eliminate this problem in its entirety by employing the resource independent PaLa [14] consensus protocol for lottery space agreement, which would require an alternate time complexity analysis. In any case, the adversary cannot attack the uniform house agreement as that is a quantum step.

C. Features of Byzantine TensorFlip

Similar to fail-stop TensorFlip (Algorithm 2), we now give the consensus features of Byzantine TensorFlip.

1) *Deterministic Distributed Consensus*: PoR blockchains like Algorand ensure safety in block proposal with a very high probability. So with a high probability all honest users will agree on the same lottery space (Algorithm 4, Step-2). Like fail-stop TensorFlip (Algorithm 2), on measurement of $|v_i^k\rangle$, whichever qudit is observed by user i will also been seen by user $j, \forall j \neq i$. This implies determinism in the house agreement (Algorithm 4, Step-3).

2) *Full-Decentralization on the House*: As an artefact of the quantum distributed house agreement, Byzantine TensorFlip also achieves full-decentralization. This is proved next.

Theorem 3 (Byzantine TensorFlip Full-Decentralization): TensorFlip-Sync-Byz achieves full-decentralization in electing $h \xleftarrow{\text{uniform}} [n]$, $b_h \xleftarrow{\text{uniform}} \{0, 1\}$.

Proof. Similar to fail-stop TensorFlip, it is easy to see that $\forall k, i \in [n], \forall a \in [n], \Pr[v_i^k = a] = \frac{1}{n}$. Next, as v_i is computed from $v_i^k, \forall k \in [n]$ under the XOR operator, as long as at least one of the set $\{v_i^k\}_{k \in [n]}$ is uniformly distributed, v_i will be uniformly distributed (for details, refer to the security of the one-time pad [15]). This implies that the house is a uniform random variable over $[n]$. Since b_h is a parity bit of a uniformly distributed bit string, b_h is also uniformly distributed. Consequently, the house election is fully decentralized. ■

3) *Expected Constant Round Complexity*: We now leverage results from blockchain consensus and quantum distributed consensus to show that Byzantine TensorFlip is a constant time protocol.

Theorem 4 (Byzantine TensorFlip Time Complexity): TensorFlip-Sync-Byz (Algorithm 4) with a Byzantine adver-

sary with size at most $\frac{n}{3}$, achieves deterministic bet space and house agreement, in expected constant time.

Proof. PoR blockchains such as Algorand achieve consensus in constant time, implying constant time lottery space agreement (Algorithm 4, Step-2). Algorithm 3 achieves consensus in expected constant time with the appropriate use of QVSS (for details please see Sec-5 in [9]), which implies constant time house agreement (Algorithm 4, Step-3). Thus the protocol terminates in expected constant time. ■

VI. DISCUSSION: TENSORFLIP UNDER ASYNCHRONY?

We now give a brief discussion on defining TensorFlip-like protocols for decentralized lotteries over an *asynchronous network*: a network model where there does not exist an upper-bound on message delivery time, and so messages can be delayed arbitrarily, or even lost forever. We will show that there are no perfect solutions as of now.

A. Limitations of Classical Consensus

The famous historical result by Fischer, Lynch and Paterson concludes that deterministic classical consensus, under asynchrony, is impossible, even with a fail-stop adversary [16]. Consequently, existing asynchronous Byzantine agreement protocols are randomized [17] [18] [19], and achieve probabilistic agreement. This is contrary to the design philosophy motivated by (synchronous) TensorFlip, which achieves deterministic agreement due to quantum entanglement.

B. Leveraging Blockchain and Quantum Consensus in the Asynchronous Setting

Similar to the protocol design in Section V, there exists a protocol construction employing blockchains and quantum consensus. To achieve lottery space agreement, the ABC asynchronous blockchain protocol [20] can be used, followed by the asynchronous version of quantum Byzantine agreement (please see Sec-6 in [9]) for fully-decentralized house agreement. Unfortunately, although this defines a secure lottery consensus, a limitation of this construction is that the bound on the adversary size is $t < \frac{n}{4}$, unlike the mandated traditional bound of $t < \frac{n}{3}$.

C. A Stepping Stone to a Fully Quantum Protocol

One possible solution direction could be to embed the bet and house proposal within vote qudits from the asynchronous QBA protocol ([9], Sec-6), as in Step-1 in the QA-Sync-Byz protocol. Unfortunately, such a protocol would be susceptible to signature forging attacks in polynomial time under a quantum adversary. However, we state that this solution idea would be stepping stone towards a fully quantum lottery, which would not employ classical verifiable random functions, but quantum signatures [21] and quantum hash functions [22].

VII. RELATED WORK

Traditional classical consensus protocols in themselves are not amenable to defining decentralized lotteries, which require agreement on a relatively large set of data corresponding to the lottery. Even from a security standpoint, classical protocols are infeasible. For example, the classical practical Byzantine fault tolerant (pBFT) protocol is susceptible to Sybil attacks [5], and thus not suitable for decentralized lottery.

Existing decentralized lotteries are mostly based on blockchain protocols, as these protocols define a reliable platform for decentralized applications (consequent to blockchains being a rapidly evolving research field [23]). Most prominent blockchain based lotteries are deployed on the Ethereum [23] decentralized platform. These include TrueFlip [24], FomoJackpot [25], Ethex Lotto [26], and FireLotto [27]. Another blockchain based lottery, DeLotto [1], has been recently proposed, which is based on the Randao [28] verifiable random number generation service. Each of the stated lotteries employ smart contracts to embed lottery rules as distributed programs.

There have also been general purpose distributed computing frameworks proposed to define decentralized lotteries. For example, Protean [29] defines distributed functional units for exposing atomic transactions to the smart contract developers. However, a time complexity analysis of Protean in the context of decentralized lotteries is missing.

Advantages of TensorFlip over Existing Lotteries

TensorFlip is superior to existing decentralized lotteries as it is not based on smart contracts, but on a faster fully-decentralized lottery agreement protocol employing blockchain consensus, and a variant of quantum distributed consensus (without the need for smart contracts, which are at a higher layer than the consensus layer in the blockchain system stack [30]), with classical verifiable random functions. Unlike smart contracts which introduce a worst case linear time complexity [31] over and above the distributed consensus step, TensorFlip is provably a constant time protocol.

VIII. CONCLUSION AND FUTURE DIRECTIONS

In this contribution, we have proposed TensorFlip, an expected constant round complexity, provably fair decentralized lottery protocol for cryptocurrency networks. Our protocol ensures that the lottery house election is not biased towards consensus resource rich users in the cryptocurrency network. Further, our protocol employs a verifiable random function for determining winnings, with a novel two-party verifiable randomness seed.

In future, we would explore the possibility of a single house proposal per user in the network. As TensorFlip currently stands as solely a theoretical construction, we would like to migrate from cryptocurrency networks for proof-of-resource blockchains, to design TensorFlip in conjunction with the

resource independent PaLa [14] and Streamlet [32] blockchain consensus protocols, and analyze the security and round complexity in this new practically realizable setting. We would also like to give a fully quantum version of TensorFlip, employing quantum signatures and quantum hash functions, to safeguard under possible attacks from a quantum polynomial time Byzantine adversary.

ACKNOWLEDGEMENTS

The author would especially like to thank Vinay J. Ribeiro (IIT Bombay) for his technical feedback and oversight on this contribution. The author would also like to thank Rajeev Shorey (IIT Delhi) and Himanshu Gandhi (IIT Delhi) for their comments to improve the readability of the paper.

REFERENCES

- [1] Z. Jia, R. Chen, and J. Li, "Delottery: A novel decentralized lottery system based on blockchain technology," in *Proceedings of the 2019 2nd International Conference on Blockchain Technology and Applications*, 2019, pp. 20–25.
- [2] W. Post, *Here's how blockchain can reduce inequality*, (Online; Accessed July 29, 2020), <https://www.washingtonpost.com/news/theworldpost/wp/2018/01/29/blockchain/>.
- [3] J. Li and W. Mann, "Initial coin offerings and platform building," 2018.
- [4] P. A. Gompers and J. Lerner, "What drives venture capital fundraising?" National bureau of economic research, Tech. Rep., 1999.
- [5] S. Bano, A. Sonnino, M. Al-Bassam, S. Azouvi, P. McCorry, S. Meiklejohn, and G. Danezis, "Sok: Consensus in the age of blockchains," in *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*, 2019, pp. 183–198.
- [6] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," Manubot, Tech. Rep., 2019.
- [7] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich, "Algorand: Scaling byzantine agreements for cryptocurrencies," in *Proceedings of the 26th Symposium on Operating Systems Principles*, 2017, pp. 51–68.
- [8] Y. Kwon, J. Liu, M. Kim, D. Song, and Y. Kim, "Impossibility of full decentralization in permissionless blockchains," in *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*, 2019, pp. 110–123.
- [9] M. Ben-Or and A. Hassidim, "Fast quantum byzantine agreement," in *Proceedings of the thirty-seventh annual ACM symposium on Theory of computing*, 2005, pp. 481–485.
- [10] E. Rieffel and W. Polak, "An introduction to quantum computing for non-physicists," *ACM Computing Surveys (CSUR)*, vol. 32, no. 3, pp. 300–335, 2000.
- [11] S. American, *Nil Communication: How to Send a Message without Sending Anything at All*, (Online; Accessed July 29, 2020), <https://www.scientificamerican.com/article/nil-communication-how-to-send-a-message-without-sending-anything-at-all/>.
- [12] P. Kwiat, J. Mitchell, P. Schwindt, and A. White, "Grover's search algorithm: an optical approach," *Journal of Modern Optics*, vol. 47, no. 2-3, pp. 257–266, 2000.
- [13] C. Crépeau, D. Gottesman, and A. Smith, "Secure multi-party quantum computation," in *Proceedings of the thirty-fourth annual ACM symposium on Theory of computing*, 2002, pp. 643–652.
- [14] T.-H. H. Chan, R. Pass, and E. Shi, "Pala: A simple partially synchronous blockchain," *IACR Cryptol. ePrint Arch.*, vol. 2018, p. 981, 2018.
- [15] J. Katz and Y. Lindell, *Introduction to modern cryptography*. CRC press, 2014.
- [16] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *Journal of the ACM (JACM)*, vol. 32, no. 2, pp. 374–382, 1985.
- [17] I. Abraham, D. Malkhi, and A. Spiegelman, "Asymptotically optimal validated asynchronous byzantine agreement," in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, 2019, pp. 337–346.

- [18] B. M. Kapron, D. Kempe, V. King, J. Saia, and V. Sanwalani, "Fast asynchronous byzantine agreement and leader election with full information," *ACM Transactions on Algorithms (TALG)*, vol. 6, no. 4, pp. 1–28, 2010.
- [19] C. Cachin, K. Kursawe, and V. Shoup, "Random oracles in constantino-ple: Practical asynchronous byzantine agreement using cryptography," *Journal of Cryptology*, vol. 18, no. 3, pp. 219–246, 2005.
- [20] J. Sliwinski and R. Wattenhofer, "Abc: Asynchronous blockchain without consensus," *arXiv preprint arXiv:1909.10926*, 2019.
- [21] D. Gottesman and I. Chuang, "Quantum digital signatures," *arXiv preprint quant-ph/0105032*, 2001.
- [22] Y.-G. Yang, P. Xu, R. Yang, Y.-H. Zhou, and W.-M. Shi, "Quantum hash function and its application to privacy amplification in quantum key distribution, pseudo-random number generation and image encryption," *Scientific reports*, vol. 6, p. 19788, 2016.
- [23] S. Leonardos, D. Reijsbergen, and G. Piliouras, "Presto: A systematic framework for blockchain consensus protocols," *IEEE Transactions on Engineering Management*, 2020.
- [24] CryptoPotato, *True Flip: The ERC-20 Decentralized Lottery Revolution*, (Online; Accessed July 29, 2020), <https://cryptopotato.com/true-flip-erc-20-decentralized-lottery-revolution/>.
- [25] Medium, *FomoJackpot: The First Truly Decentralized Lottery Platform*, (Online; Accessed July 29, 2020), <https://medium.com/dabanking.io/introducing-fomojackpot-the-first-truly-\decentralized-lottery-platform-a7662bc79ce1>.
- [26] EthexLotto, *Ethex Lotto: A Crypto-Lottery Game*, (Online; Accessed July 29, 2020), <https://ethex.bet/>.
- [27] FireLotto, *Fire Lotto: An International Blockchain Lottery*, (Online; Accessed July 29, 2020), <https://firelotto.io/>.
- [28] R. W. 2017), *Randao: Verifiable Random Number Generation*, (Online; Accessed July 29, 2020), <https://www.randao.org/whitepaper/RandaoV0.85en.pdf>.
- [29] E. C. Alp, E. Kokoris-Kogias, G. Fragkoulis, and B. Ford, "Rethinking general-purpose decentralized computing," in *Proceedings of the Workshop on Hot Topics in Operating Systems*, 2019, pp. 105–112.
- [30] W. Wang, D. T. Hoang, P. Hu, Z. Xiong, D. Niyato, P. Wang, Y. Wen, and D. I. Kim, "A survey on consensus mechanisms and mining strategy management in blockchain networks," *IEEE Access*, vol. 7, pp. 22 328–22 370, 2019.
- [31] K. Chatterjee, A. K. Goharshady, and E. K. Goharshady, "The treewidth of smart contracts," in *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, 2019, pp. 400–408.
- [32] B. Y. Chan and E. Shi, "Streamlet: Textbook streamlined blockchains," *IACR Cryptol. ePrint Arch.*, vol. 2020, p. 88, 2020.